

Mari Ozuna Garcia

Jared Nepomuceno

CS122A

Professor Allan Knight

9 June 2026

CS122 Custom Lab Report

Dual-Processor Hardware MIDI Sequencer and Synthesizer

Introduction

For our project, we built a hardware MIDI step sequencer and synthesizer using two processors working together: a Raspberry Pi Pico microcontroller and an IceSugar-Pro FPGA board. A user can program a short musical loop by turning notes on and off on a 16x8 grid, then play it back in real time while controlling the speed with a knob.

The Pico handles all the user input such as the reading of the joystick, button, and BPM potentiometer and keeps track of which notes are active. It sends that information to the FPGA over SPI. The FPGA is responsible of the visual display, driving a 4.3-inch RGB LCD screen that shows the grid, a moving playhead, and the cursor position. It also plays audio through a passive piezo buzzer using a PWM square wave.

All planned features were implemented. The grid display, playhead animation, joystick navigation, note toggling, BPM adjustment, and audio output are all fully working.

Elements of Complexity

- Dual-processor design: The Pico and FPGA each have their own job and communicate in real time over SPI. Splitting the work across two chips meant we had to design a clean protocol between them and handle timing carefully. This was fully implemented.
- Custom SPI protocol: We designed our own 2-byte command format. The first byte tells the FPGA what kind of message it is (note on, note off, grid update, or cursor move). The second byte carries the data (like which column or row mask to update). This was fully implemented.
- LCD video generation in Verilog: The FPGA generates the pixel clock, DEN, and pixel color data entirely in hardware. We use no external framebuffer chip or library. The 480x272 display is driven directly from the FPGA with a scan-line pixel counter. This was fully implemented.

- Audio synthesis in hardware: The FPGA uses a clock divider approach to generate square waves at exact musical frequencies (C4 through C5) using only a counter and toggle register. No DAC or audio library is used. This was fully implemented.
- Cooperative task scheduler on the Pico: Instead of running three separate FreeRTOS tasks, we built a lightweight cooperative scheduler inside a single FreeRTOS task. Three state-machine tick functions (JS_Tick, Tick_Sequencer, LED_Tick) are dispatched by a period-based loop running every 10 ms (GCD_PERIOD). Each tick function fires only when its own period has elapsed: input at 50 ms, sequencer at 10 ms, LED at 10 ms. Because all three run sequentially in the same task, no mutex is needed on the SPI bus. This was fully implemented.
- Metastability protection: Because SPI signals come from a different clock domain, the FPGA uses 2-3 stage synchronizer flip-flops on all incoming SPI lines to prevent glitches. This was fully implemented.

User Guide

Here is how to use the sequencer:

- Power on both the Pico and the FPGA board. The LCD will turn on and show an 8-row by 16-column grid. All cells start empty (no notes).
- Use the joystick to move the cursor around the grid. Push left/right to move between columns (steps 1-16). Push up/down to move between rows (notes C4 through C5).
- Press the button to toggle a note on or off at the cursor position. Active notes show up as a bright yellow-green block in the cell. The FPGA display updates immediately.
- Turn the potentiometer knob to change the playback speed (BPM). Turning it all the way down gives about 40 BPM; all the way up gives about 140 BPM.
- The sequencer plays automatically in a loop. A yellow column highlight (the playhead) moves left to right across the screen to show which step is currently playing.
- When the playhead lands on a step that has an active note, the buzzer plays the corresponding pitch. Each row maps to a musical note: row 1 = C5, row 2 = B4, row 3 = A4, row 4 = G4, row 5 = F4, row 6 = E4, row 7 = D4, row 8 = C4.
- The onboard LED on the Pico blinks in sync with the BPM as a visual tempo indicator.

Hardware Components Used

- Raspberry Pi Pico 2 (RP2350 microcontroller)
- IceSugar-Pro FPGA development board (Lattice ECP5)
- 4.3" RGB LCD display (480x272, parallel RGB interface)
- Passive piezo buzzer
- 10kΩ potentiometer (for BPM control)
- 2-axis analog joystick module
- Tactile push button
- Jumper wires and breadboard

Software Libraries Used

- FreeRTOS (for the Raspberry Pi Pico / RP2350): Used to run three concurrent tasks on the Pico: input polling, sequencer tick timing, and the LED heartbeat. The FreeRTOS kernel manages task switching and a mutex protects shared SPI bus access.
- Pico SDK (pico/stdlib.h, hardware/adc.h, hardware/spi.h, pico/cyw43_arch.h): Used to configure SPI, read ADC inputs (potentiometer and joystick), control GPIO pins, and drive the onboard CYW43 LED. Provides all the low-level hardware abstraction for the RP2350.

Protocols Used

- SPI (Serial Peripheral Interface): The Pico acts as the SPI master and the FPGA acts as the SPI slave. The Pico sends 2-byte messages over SPI at 1 MHz to update the FPGA's note grid, advance the playhead, and move the cursor. SPI was chosen because it is fast, simple, and easy to implement in both C and Verilog.

How We Met the Requirements

The following requirements were listed in our proposal:

1. Sequence Engine (Pico / FreeRTOS)

We ended up not using three separate FreeRTOS tasks like we originally planned. Instead we wrote a cooperative scheduler inside one FreeRTOS task called `scheduler_task`. It loops every 10 ms and calls each tick function when that function's timer runs out. `JS_Tick` runs every 50 ms and handles the joystick, button, and

potentiometer. Tick_Sequencer runs every 10 ms and figures out when to send NOTE_ON and NOTE_OFF to the FPGA based on the current BPM. LED_Tick also runs every 10 ms and handles the heartbeat LED. A shared grid array in RAM keeps track of which notes are active, and a column bitmask gets sent to the FPGA whenever a note changes. Since all three functions run one at a time inside the same task, we never need a mutex for the SPI bus.

2. Visual Display (FPGA / Verilog)

The FPGA drives the 480x272 LCD directly using pixel counters in Verilog. It draws a 16x8 grid made up of 28x24 pixel cells, highlights the current playhead column in yellow, shows the cursor position with a teal outline, and fills in a bright block for any active notes. All of this happens in a combinational always block that decides what color to output for each pixel based on where the scan counters are at that moment. The note data comes from a 16-entry note_grid register array that the FPGA updates whenever it receives a grid update command over SPI.

3. Audio Synthesis (FPGA / Verilog)

For audio, the FPGA has a square wave generator that counts clock cycles and toggles the buzzer pin at the right frequency for each note. We precomputed the half-period (in clock cycles) for all 8 notes from C4 to C5 and stored them in a case statement. When the Pico sends a NOTE_ON, the FPGA looks at the current playhead column, finds the first active row in that column using a priority encoder, looks up the half-period for that row, and starts toggling the buzzer output at that rate. When NOTE_OFF comes in, the buzzer goes silent.

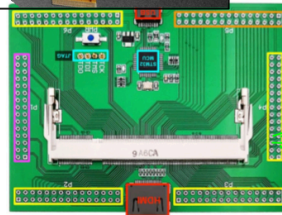
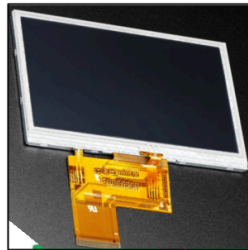
4. User Interaction

The joystick is handled by JS_Tick with a simple three-state machine (JS_Start, JS_Idle, JS_Moved). When you push the joystick, it moves into JS_Moved and fires the cursor update exactly once by checking if the direction changed from the previous tick. When you let go, it goes back to JS_Idle and waits. This prevents the cursor from running away if you hold the joystick. The button is read every 50 ms and toggles the note at the current cursor position on the falling edge, then immediately sends a GRID_UPDATE to the FPGA so the screen reflects the change. The potentiometer is also read every 50 ms and mapped to a BPM between 40 and 140, which Tick_Sequencer uses to recalculate step timing on every loop.

Wiring Diagram

FPGA → LCD (FPC connector, internal):

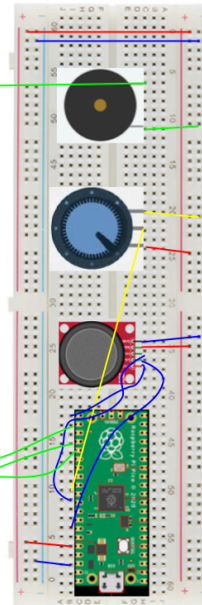
LCD_CLK (J13) → LCD clock
LCD_DEN (F13) → LCD data enable
LCD_R[0] (P11) → LCD red bit 0
LCD_R[1] (N12) → LCD red bit 1
LCD_R[2] (N13) → LCD red bit 2
LCD_R[3] (M12) → LCD red bit 3
LCD_R[4] (L14) → LCD red bit 4
LCD_G[0] (P12) → LCD green bit 0
LCD_G[1] (P13) → LCD green bit 1
LCD_G[2] (P14) → LCD green bit 2
LCD_G[3] (M13) → LCD green bit 3
LCD_G[4] (L13) → LCD green bit 4
LCD_G[5] (K13) → LCD green bit 5
LCD_B[0] (K14) → LCD blue bit 0
LCD_B[1] (J14) → LCD blue bit 1
LCD_B[2] (H14) → LCD blue bit 2
LCD_B[3] (G14) → LCD blue bit 3
LCD_B[4] (F14) → LCD blue bit 4



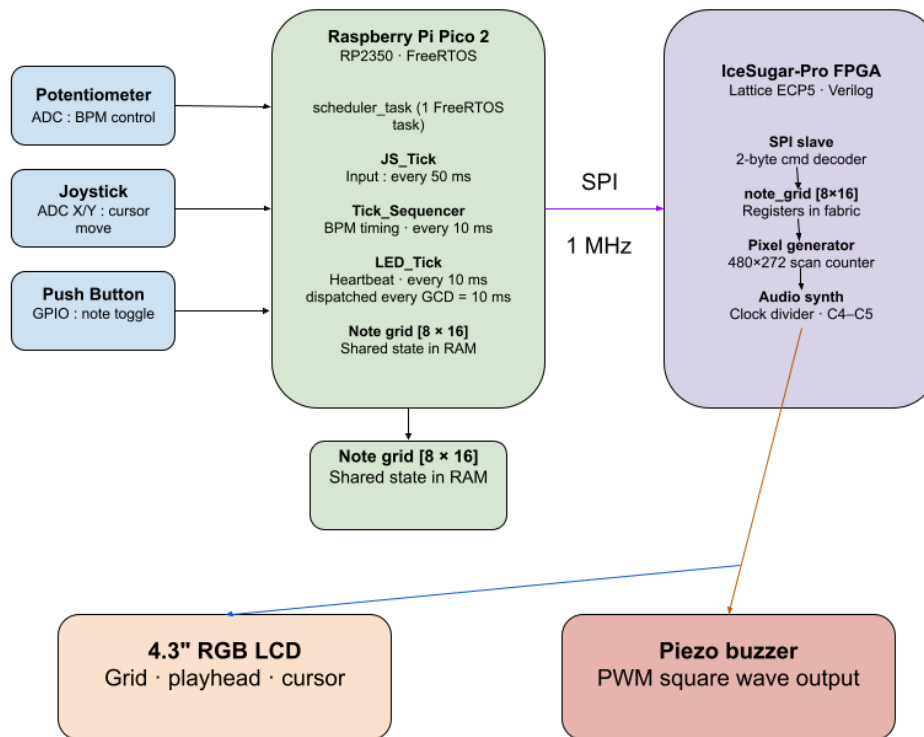
FPGA → Buzzer:

BUZZER (ball C4, ULX3S GP10)
→ Piezo (+)
Any FPGA GND → Piezo (-)

- Pico GPIO 18 (SCK) → FPGA SPI_SCK
- Pico GPIO 19 (MOSI) → FPGA SPI_MOSI
- Pico GPIO 17 (CS) → FPGA SPI_CS
- Pico GPIO 26 (ADC0) → Potentiometer wiper
- Pico GPIO 27 (ADC1) → Joystick Y-axis
- Pico GPIO 28 (ADC2) → Joystick X-axis
- Pico GPIO 16 → Tactile button (pull-up enabled)



Design Diagram



AI Usage

We used AI (Claude) to help with parts of this project. Specifically, we used it to help debug the FreeRTOS task structure on the Pico, get guidance on the SPI synchronizer design in Verilog (the 2-3 stage flip-flop chain for metastability protection), and to help calculate the correct half-period values for each musical note given the FPGA clock frequency. AI also helped us organize this report. Also we used it to get guidance to make helper functions that are needed for this project.

Acknowledgements

- FreeRTOS documentation:
<https://www.freertos.org/Documentation/02-Kernel/03-Supported-devices/02-Customization>
- Raspberry Pi Pico SDK documentation and examples:
<https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf>

- 2-Step SPI Protocol:
https://www.raspberrypi.com/documentation/pico-sdk/hardware.html#hardware_spi

Documented in: pico/pico.c#L43-L51

- IceSugar-Pro FPGA board schematic and pinout documentation from CS122A
- ANSI/EIA RS-274D standard for step sequencer grid conventions (background reference)
- CS 122A course materials and lab documentation, UC Riverside